ARCHWAY

Architecture + Pathway

The Bridge Between Thinking and Building

Pre-Coding Documentation — Version 2.0 (Post-Brainstorm Revision)

Field	Detail
Project Name	Archway
Version	2.0 (Revised with refined USP + Task Generation)
Author	Iqbal
Date	March 2026
Status	Sprint Zero — Requirements Locked (v2)
Tagline	Better input, better output. Think through everything before you build anything.

Daftar Isi

1. Executive Summary & USP
2. Pain Points & Solutions (Quick Reference)
3. Problem Statement & Vision
4. Target Users & Personas
5. Competitive Landscape
6. Functional Requirements
7. Non-Functional Requirements
8. User Stories
9. Use Case Diagram
10. User Flow (Flowcharts)
11. Data Flow Diagram (DFD)
12. Entity Relationship Diagram (ERD)
13. Sequence Diagrams
14. Task Generation Engine (Core Differentiator)
15. Tech Stack
16. Sprint Plan & Roadmap
17. Definition of Done
18. Wireframe Notes

Appendix A: Pain Point → Solution Detailed Mapping

1. Executive Summary & USP

Archway adalah web application yang memandu developer melewati seluruh tahap pre-coding secara visual menggunakan node-based pipeline, lalu menghasilkan actionable task list yang siap digunakan di project management tool manapun.

Unique Selling Proposition

> **"Archway is the bridge between thinking and building — a guided visual workspace that turns your pre-coding documentation into actionable tasks, so that whatever comes next (Claude Code, manual coding, Linear, Notion) works better."**

Positioning

Archway bukan pengganti Excalidraw, Linear, atau Claude Code. Archway adalah layer yang duduk SEBELUM semua tools tersebut:

```
┌──────────────────────────────────┐
│            ARCHWAY               │
│  Guided node-based pre-coding    │
│  documentation workspace         │
│            ↓                     │
│  Auto-generated task list        │
│            ↓                     │
│  Export: MD / CSV / JSON / PDF   │
└──────────────┬───────────────────┘
               │
   ┌───────────┼──────────┐
   ↓           ↓          ↓
 Linear     Notion    GitHub Issues
   ↓           ↓          ↓
 Claude     Manual     Cursor
 Code       Coding     / IDE
```

Core Value Loop

Archway menyelesaikan 3 masalah sekaligus dalam satu flow:

Step	What Archway Does	Value
1. THINK	Guided node-by-node documentation (Requirements → User Stories → Flowchart → ERD → etc)	Developer dipandu untuk berpikir lengkap, nggak ada yang terlewat
2. PLAN	Auto-generate task list dari semua dokumen yang sudah dibuat	Docs langsung jadi actionable tasks, no double work
3. SHIP	Export ke format apapun (MD/CSV/JSON/PDF) untuk tool apapun	Siap masuk ke Linear, Notion, GitHub, atau langsung coding

2. Pain Points & Solutions (Quick Reference)

Tabel ini adalah ringkasan cepat mengapa Archway ada. Detail lengkap di Appendix A.

Pain Point	Problem	Archway Solution
P1	Scope creep — fitur nambah terus karena nggak ada batasan jelas di awal	Scope di-lock di Project Brief & Requirements node. Perubahan = conscious decision
P2	Kehilangan arah di tengah project, lupa mau ngerjain apa	Visual pipeline = peta. Task Board = checklist. Selalu ada anchor
P3	Database asal-asalan, harus refactor berkali-kali	ERD node memaksa mikir data structure SEBELUM coding
P4	Tools dokumentasi terpisah-pisah (ERD di sini, flowchart di sana)	Satu workspace: semua diagram + docs di satu tempat
P5	Nggak tahu harus mulai dari mana bikin dokumentasi	Guided pipeline node-by-node dengan recommended order
P6	Dokumentasi terasa boring dan buang waktu	Node-based visual experience, bukan blank document
P7	Hasil dokumentasi nggak actionable (cuma pajangan)	Auto task generation: docs → langsung jadi task list
P8	Harus translate manual dari doc ke task list (double work)	Progressive task suggestion seiring mengisi setiap node
P9	Susah estimasi effort karena nggak ada breakdown jelas	Granular task breakdown dari semua diagram types
P10	Nggak ada source of truth saat coding	Archway = single source of truth, exportable ke mana aja

3. Problem Statement & Vision

3.1 Problem Statement

Developer, terutama solo developer dan freelancer, menghadapi tiga masalah yang saling terhubung:

Problem 1: Skipping Pre-Coding (The Vibe Coding Trap)

Developer langsung loncat ke coding tanpa dokumentasi. Hasilnya: edge case baru ketahuan di tengah jalan, database harus direfactor, scope creep, dan kehilangan arah.

Problem 2: Fragmented & Boring Documentation Tools

Tools yang ada (Excalidraw, Draw.io, Notion) terpisah-pisah, nggak ada guided flow, dan terasa seperti mengerjakan tugas kampus. Hasilnya: developer malas bikin dokumentasi.

Problem 3: Documentation ≠ Action

Bahkan kalau developer bikin dokumentasi, hasilnya sering jadi pajangan. Nggak ada jembatan otomatis dari dokumen ke task list. Developer harus manual translate PRD ke tasks (double work), dan sering task-nya nggak lengkap karena PRD-nya kurang detail.

3.2 Vision

> **"Satu platform visual yang memandu developer melewati seluruh tahap pre-coding, menghasilkan dokumentasi lengkap DAN actionable task list yang siap dipakai di tool manapun — menjadikan proses pre-coding engaging, bukan membosankan."**

3.3 Success Metrics

Metric	Target	Cara Ukur
Core flow completion	Create project → isi nodes → generate tasks → export dalam < 5 menit onboarding	Manual testing
Task generation accuracy	80%+ generated tasks are relevant (user keeps them, nggak delete)	User testing / analytics
Export usability	Exported tasks bisa di-import ke Linear/Notion tanpa reformatting	Integration testing
Data persistence	Zero data loss across browser sessions	Automated test

4. Target Users & Personas

Persona 1: Solo Developer / Freelancer

Aspek	Detail
Background	Developer dengan pengalaman kerja & freelance, nyaman coding tapi sering skip dokumentasi
Behavior	Sering vibe coding. Pakai Claude Code / Cursor. Tahu Linear/Notion tapi nggak konsisten pakai.
Pain Points	P1 (scope creep), P2 (kehilangan arah), P3 (database refactor), P7 (docs nggak actionable)

Goal	Proses pre-coding yang terstruktur, engaging, dan langsung menghasilkan task list
Success	"Aku bisa bikin dokumentasi lengkap + task list dalam 1 jam sebelum coding"

Persona 2: Vibe Coder with AI Tools

Aspek	Detail
Background	Developer yang aktif pakai Claude Code / Cursor untuk coding. Tahu workflow PRD → Claude Code → Linear.
Behavior	Bikin PRD manual (kualitas bervariasi), lalu kasih ke AI. Hasil tergantung kualitas input.
Pain Points	P5 (nggak tahu mulai dari mana), P8 (double work translate docs → tasks), P9 (estimasi susah)
Goal	Tool yang bikin PRD-nya jadi lebih bagus, sehingga output AI juga lebih bagus
Success	"PRD dari Archway bikin Claude Code generate tasks yang jauh lebih lengkap dan akurat"

Persona 3: Junior Developer / CS Student

Aspek	Detail
Background	Mahasiswa CS atau developer junior yang baru belajar SDLC
Pain Points	P5 (nggak tahu mulai dari mana), P6 (dokumentasi boring)
Goal	Tool yang memandu step-by-step, educational sekaligus practical
Success	"Aku akhirnya paham cara bikin ERD dan flowchart yang benar, dan langsung bisa coding dari situ"

5. Competitive Landscape

Mapping tools yang ada dan gap yang Archway isi.

Tool	Strengths	Gap (yang Archway isi)
Excalidraw / Draw.io	Visual diagram editor, free, collaborative	Nggak guided. Nggak generate tasks. Terpisah dari docs lain.
Notion	Flexible docs + database, templates	Nggak ada guided SDLC flow. Nggak auto-generate tasks dari diagrams.
Linear / Jira	Excellent task management, sprints, tracking	Nggak ada pre-coding documentation layer. Assume PRD sudah ada.

Claude Code + Linear MCP	AI generate tasks dari PRD, push ke Linear	PRD quality depends on developer. Nggak visual. Nggak guided.
Mermaid.js	Diagram as code, embeddable	Just a rendering library. No workflow, no task generation.

Archway's Unique Position

Archway is the only tool that combines: (1) guided pre-coding documentation, (2) all diagram types in one workspace, (3) auto task generation from docs, and (4) format-agnostic export to any PM tool.

6. Functional Requirements

6.1 Dashboard & Project Management

ID	Requirement	Priority	Solves
FR-01	User dapat melihat daftar semua project di dashboard	Must	P2
FR-02	User dapat membuat project baru (nama + deskripsi)	Must	—
FR-03	User dapat rename project	Must	—
FR-04	User dapat delete project (dengan konfirmasi)	Must	—
FR-05	Dashboard menampilkan progress per project (berapa node Done)	Should	P2
FR-06	Dashboard menampilkan last modified date per project	Should	—
FR-07	User dapat duplicate project	Could	—

6.2 Node Pipeline Editor

ID	Requirement	Priority	Solves
FR-10	Project workspace menampilkan node-based pipeline (React Flow)	Must	P4, P6
FR-11	Default template: Brief → Requirements → User Stories → Use Case → Flowchart → DFD → ERD → Sequence → Task Board → Summary	Must	P5
FR-12	User dapat menambah custom node ke pipeline	Must	—
FR-13	User dapat menghapus node dari pipeline	Must	—
FR-14	User dapat mereorder/memindahkan node (drag-and-drop)	Must	—
FR-15	Setiap node menampilkan status: Empty / In Progress / Done	Must	P2

FR-16	Soft guidance: recommended next node di-highlight, tapi nggak di-lock	Must	P5
FR-17	Pipeline supports zoom dan pan	Must	—

6.3 Node Editors

ID	Requirement	Priority	Solves
FR-20	Klik node membuka editor panel di samping kanan (drawer/side panel)	Must	—
FR-21	Setiap node type memiliki editor yang sesuai	Must	—
FR-22	Editor hybrid: guided fields (form) + free-text area	Must	P5, P6
FR-23	Project Brief: nama, deskripsi, scope, target user, tech stack	Must	P1
FR-24	Requirements: list + priority (MoSCoW)	Must	P1
FR-25	User Stories: As a [role], I want [X], so that [Y]	Must	P7
FR-26	Diagram nodes: Mermaid editor + live preview + starter templates	Must	P3, P5
FR-27	Summary node: auto-compile dari semua node, read-only	Must	P10
FR-28	Auto-save saat user mengetik (debounced 500ms)	Must	—
FR-29	Node status auto-update: Empty → In Progress → Done (manual toggle)	Must	P2

6.4 Task Generation Engine

ID	Requirement	Priority	Solves
FR-30	Progressive task suggestion: tasks di-suggest seiring user mengisi setiap node	Must	P7, P8
FR-31	Rule-based extraction: User Stories → tasks, ERD entities → DB tasks, Flowchart processes → implementation tasks, Sequence messages → API tasks	Must	P8, P9
FR-32	Dedicated Task Board node di pipeline menampilkan semua generated tasks	Must	P2, P7
FR-33	Flexible grouping: user bisa pilih group by User Story / Layer / Feature Flow	Must	—
FR-34	User bisa edit/modify generated tasks	Must	—
FR-35	User bisa delete tasks yang nggak relevan	Must	—
FR-36	User bisa add manual tasks di atas generated tasks	Must	—
FR-37	User bisa re-generate tasks setelah update dokumen	Must	—
FR-38	Setiap task menampilkan source (dari node mana task ini di-generate)	Should	P10

6.5 Export

ID	Requirement	Priority	Solves
FR-40	Export seluruh project sebagai PDF (text + rendered diagrams)	Must	P10
FR-41	Export seluruh project sebagai Markdown	Must	P10
FR-42	Export task list sebagai Markdown checklist (- [ ] format)	Must	P7, P8
FR-43	Export task list sebagai CSV (importable ke Linear/Notion/spreadsheet)	Must	P7, P8
FR-44	Export task list sebagai JSON (consumable by API/Claude Code)	Must	P7, P8
FR-45	Export task list dalam Linear-compatible CSV format	Should	P8
FR-46	Export individual node/diagram sebagai PNG	Should	—

6.6 Data Persistence

ID	Requirement	Priority	Solves
FR-50	Semua data disimpan di IndexedDB via Dexie.js	Must	—
FR-51	Data persist across browser sessions	Must	—
FR-52	Schema mendukung future migration ke cloud sync	Should	—

7. Non-Functional Requirements

Category	Requirement
Performance	App load < 3 detik. Pipeline responsif tanpa lag saat 10+ nodes. Task generation < 2 detik.
Usability	User baru bisa create project dan mulai mengisi node dalam < 2 menit tanpa tutorial.
Reliability	Zero data loss. Auto-save reliable. Mermaid errors handled gracefully.
Compatibility	Chrome, Firefox, Safari, Edge terbaru. Desktop-first, responsive tablet.
Scalability	Handle 50+ projects, 15+ nodes per project tanpa degradation.
Maintainability	Modular codebase. Setiap node type = separate component. Task extraction rules = pluggable.

8. User Stories

Epic 1: Project Management

ID	User Story	Acceptance Criteria
US-01	As a developer, I want to create a new project so that I can start documenting.	New Project → isi nama → redirect ke workspace with default pipeline.
US-02	As a developer, I want to see all projects on a dashboard so I can access any project.	Dashboard shows project cards with name, progress bar, last modified.
US-03	As a developer, I want to delete a project with confirmation.	Delete → confirm dialog → removed from DB and dashboard.

Epic 2: Node Pipeline

ID	User Story	Acceptance Criteria
US-10	As a developer, I want a visual pipeline so I know what to do before coding.	React Flow canvas with default nodes connected sequentially.
US-11	As a developer, I want each node to show status so I can track progress.	Badges: Empty (gray), In Progress (yellow), Done (green).
US-12	As a developer, I want to add custom nodes for project-specific docs.	Click + → choose type or custom → new node on canvas.
US-13	As a developer, I want soft guidance on recommended order.	Recommended next node has subtle glow. All nodes remain clickable.

Epic 3: Node Editing

ID	User Story	Acceptance Criteria
US-20	As a developer, I want to click a node and edit in a side panel.	Click node → drawer opens right → pipeline visible left.
US-21	As a developer, I want guided fields per node type.	Each node type has specific fields + Additional Notes textarea.
US-22	As a developer, I want Mermaid live preview for diagram nodes.	Split: syntax editor + rendered preview. Updates on keystroke (debounced).
US-23	As a developer, I want auto-save so I never lose work.	Debounce 500ms → save to IndexedDB. No manual save needed.

Epic 4: Task Generation (Core Differentiator)

ID	User Story	Acceptance Criteria
US-30	As a developer, I want tasks progressively suggested as I fill nodes so I see value immediately.	After saving a node, relevant tasks appear in Task Board with source label.
US-31	As a developer, I want to choose how tasks are grouped.	Toggle: group by User Story / Layer / Feature Flow. Instant re-group.
US-32	As a developer, I want full control: edit, delete, add manual tasks.	Each task has edit/delete buttons. "+" button for manual tasks.
US-33	As a developer, I want to re-generate tasks after updating docs.	"Regenerate" button. Confirms before overwriting. Keeps manual tasks.
US-34	As a developer, I want each task to show its source node.	Task card shows badge: "from User Stories" or "from ERD" etc.

Epic 5: Export

ID	User Story	Acceptance Criteria
US-40	As a developer, I want to export entire project as PDF.	Export PDF → all content + rendered diagrams → download.
US-41	As a developer, I want to export as Markdown for my repo.	Export MD → text + Mermaid code blocks → download .md file.
US-42	As a developer, I want to export task list in multiple formats.	Export tasks as: MD checklist, CSV, JSON, Linear CSV. All downloadable.
US-43	As a developer, I want to export individual diagrams as PNG.	Each diagram node has Export PNG button.

9. Use Case Diagram

Actor	Description
Developer (Primary)	Creates projects, fills documentation, manages tasks, exports
System (Browser)	Handles auto-save, rendering, task generation, storage

Use Cases (Mermaid-ready)

```
Developer:
  — Create / Edit / Delete Project
  — Navigate Pipeline
  — Edit Node Content
      «include» Fill Guided Fields
      «include» Write Mermaid Syntax (diagram nodes)
      «extend» Add Free-text Notes
  — Add / Remove Custom Node
  — Preview Diagram
  — View Task Board
      «include» View Generated Tasks
      «extend» Edit / Delete / Add Tasks
      «extend» Change Grouping Mode
      «extend» Regenerate Tasks
  — View Summary
  — Export Project (PDF / MD)
  — Export Task List (MD / CSV / JSON / Linear CSV)
      «extend» Export Individual Diagram (PNG)

System:
  — Auto-save to IndexedDB
  — Render Mermaid Diagrams
  — Progressive Task Generation (triggered per node save)
  — Update Node Status
  — Compile Summary
```

10. User Flow (Flowcharts)

Flow 1: Create New Project

```
flowchart TD
    A[User buka Archway] --> B[Dashboard]
    B --> C[Klik New Project]
    C --> D[Modal: Input Nama + Deskripsi]
    D --> E{Nama diisi?}
    E -- Tidak --> F[Validasi error] --> D
    E -- Ya --> G[Simpan ke IndexedDB]
    G --> H[Generate default pipeline nodes + Task Board]
    H --> I[Redirect ke Project Workspace]
```

Flow 2: Edit Node + Progressive Task Generation

```
flowchart TD
    A[User klik Node] --> B[Side panel opens]
    B --> C{Node type?}
    C -- Text --> D[Show guided fields + free-text]
    C -- Diagram --> E[Show Mermaid editor + preview]
    D --> F[User edits content]
    E --> F
    F --> G[Auto-save, debounce 500ms]
    G --> H[Update node status]
    H --> I[Task Generation Engine]
    I --> J[Extract tasks from this node using rules]
    J --> K[Add/update tasks in Task Board node]
    K --> L{User clicks another node?}
    L -- Yes --> A
    L -- Close --> M[Full pipeline view]
```

Flow 3: Task Board Interaction

```
flowchart TD
    A[User opens Task Board node] --> B[Show all generated + manual tasks]
    B --> C{User action?}
    C -- Change grouping --> D[Regroup: by Story / Layer / Flow]
    C -- Edit task --> E[Inline edit task details]
    C -- Delete task --> F[Confirm + remove]
    C -- Add manual --> G[New task form]
    C -- Regenerate --> H{Confirm overwrite?}
    H -- Yes --> I[Re-run extraction on all nodes]
    I --> J[Merge: keep manual, update generated]
    H -- No --> B
    C -- Export --> K{Choose format}
    K --> L[MD / CSV / JSON / Linear CSV]
    L --> M[Download file]
```

Flow 4: Export Project

```
flowchart TD
    A[User klik Export] --> B{Choose type}
    B -- Full PDF --> C[Compile all nodes + render diagrams]
    C --> D[Generate PDF] --> E[Download]
    B -- Full Markdown --> F[Compile nodes + Mermaid blocks]
    F --> G[Generate .md] --> E
    B -- Tasks Only --> H{Choose format}
    H --> I[MD checklist / CSV / JSON / Linear CSV]
    I --> E
```

11. Data Flow Diagram (DFD)

Level 0: Context Diagram

```
[Developer]
    -- project data, node content, task edits -->
        (Archway System)
    <-- rendered diagrams, task list, exported documents --
```

```
            (Archway System)
        -- read/write --> [IndexedDB Storage]
```

Level 1: Sub-processes

Process	Input	Output	Data Store
1.0 Project Mgmt	Project name, description	Project list, details	DB: Projects
2.0 Node Editor	User text, Mermaid syntax	Saved content, status	DB: Nodes, NodeContents
3.0 Diagram Renderer	Mermaid syntax	SVG/image, errors	None (stateless)
4.0 Pipeline Manager	Node statuses, edges	Visual pipeline, guidance	DB: Edges
5.0 Task Engine	All node contents	Generated tasks	DB: Tasks
6.0 Export Engine	All data + rendered diagrams	PDF, MD, CSV, JSON	None (on demand)

12. Entity Relationship Diagram (ERD)

Mermaid ERD

```
erDiagram
    PROJECTS ||--|{ NODES : contains
    PROJECTS ||--|{ EDGES : contains
    NODES ||--o| NODE_CONTENTS : has
    NODES ||--o{ TASKS : generates

    PROJECTS {
        string id PK
        string name
        string description
        datetime created_at
        datetime updated_at
    }

    NODES {
        string id PK
        string project_id FK
        string type
        string label
        string status
        float position_x
        float position_y
        int sort_order
        datetime updated_at
    }

    NODE_CONTENTS {
```

```
        string id PK
        string node_id FK
        json guided_fields
        string free_text
        string mermaid_syntax
        datetime updated_at
    }

    EDGES {
        string id PK
        string project_id FK
        string source_node_id FK
        string target_node_id FK
    }

    TASKS {
        string id PK
        string project_id FK
        string source_node_id FK
        string title
        string description
        string group_key
        string priority
        string labels
        boolean is_manual
        int sort_order
        datetime created_at
        datetime updated_at
    }
```

New Entity: TASKS

Field	Type	Description
id	string (UUID)	Primary key
project_id	string (FK)	Referensi ke PROJECTS.id
source_node_id	string (FK)	Node yang men-generate task ini (null jika manual)
title	string	Judul task, e.g. "Create Users table migration"
description	string	Detail task, optional
group_key	string	Key untuk grouping: story ID, layer name, atau feature name
priority	enum	must / should / could / wont (inherited from requirement priority)
labels	JSON string	Array of labels: ["frontend", "auth", "database"]
is_manual	boolean	true = user-created. false = auto-generated. Manual tasks preserved on regenerate.
sort_order	int	Urutan tampilan dalam group
created_at	datetime	Timestamp
updated_at	datetime	Timestamp

Relationships

- PROJECTS → NODES: 1:N (satu project punya banyak nodes)
- PROJECTS → EDGES: 1:N (satu project punya banyak edges)
- NODES → NODE_CONTENTS: 1:1 (setiap node punya tepat satu content)
- NODES → TASKS: 1:N (satu node bisa generate banyak tasks)
- PROJECTS → TASKS: 1:N (satu project punya banyak tasks, termasuk manual)

Design Decisions

- NODE_CONTENTS dipisah dari NODES: content bisa besar, pipeline rendering nggak perlu load semua content.
- TASKS punya source_node_id: supaya user tahu task ini dari mana (traceability). Juga dibutuhkan saat regenerate.
- is_manual flag: saat regenerate, hanya tasks dengan is_manual=false yang di-replace. Manual tasks aman.

13. Sequence Diagrams

Sequence 1: Edit Node + Progressive Task Generation

```
sequenceDiagram
    actor Dev as Developer
    participant Canvas as React Flow
    participant Panel as Editor Panel
    participant Store as Zustand Store
    participant DB as IndexedDB
    participant Mermaid as Mermaid.js
    participant TaskEng as Task Engine
    participant TaskBoard as Task Board

    Dev->>Canvas: Click node
    Canvas->>Store: setActiveNode(nodeId)
    Store->>DB: nodeContents.get(nodeId)
    DB-->>Panel: content data

    Dev->>Panel: Edit content
    Panel->>Panel: Debounce 500ms
    Panel->>DB: save content

    alt Diagram Node
        Panel->>Mermaid: render(syntax)
        Mermaid-->>Panel: SVG or error
    end

    Panel->>TaskEng: extractTasks(nodeType, content)
    TaskEng->>TaskEng: Apply extraction rules
    TaskEng->>DB: upsert generated tasks (preserve manual)
    TaskEng-->>TaskBoard: Updated task list
    Store-->>Canvas: Update node status badge
```

Sequence 2: Export Task List

```
sequenceDiagram
    actor Dev as Developer
    participant TB as Task Board
    participant Store as Zustand
    participant DB as IndexedDB
    participant Exp as Export Engine

    Dev->>TB: Click Export Tasks
    TB->>Dev: Show format picker (MD/CSV/JSON/Linear)
    Dev->>TB: Select CSV
    TB->>Store: getTasksByProject(projectId)
    Store->>DB: tasks.where({project_id}).toArray()
    DB-->>Store: all tasks
    Store->>Exp: formatAsCSV(tasks, grouping)
    Exp-->>Dev: Download tasks.csv
```

14. Task Generation Engine (Core Differentiator)

Ini adalah fitur yang membedakan Archway dari semua tool lain. Detail teknis tentang bagaimana tasks di-generate dari setiap node type.

14.1 Architecture Overview

```
Node saved
    ↓
Task Engine receives: { nodeType, content }
    ↓
Apply extraction rules based on nodeType
    ↓
Generate task objects with source_node_id
    ↓
Upsert to DB (match by source + rule key)
    ↓
Task Board auto-updates (preserve manual tasks)
```

14.2 Extraction Rules per Node Type

Source Node	Extraction Rule	Generated Task Example
User Stories	Each user story → 1 task. Priority inherited from Requirements.	"Implement user registration" (Must)
ERD	Each entity → 1 DB task. Each relationship → 1 migration note.	"Create Users table + model" with subtask list of fields
Flowchart	Each process node → 1 task. Each decision → 1 validation task.	"Build checkout form UI", "Validate stock availability"
Sequence Diagram	Each cross-component message → 1 API/integration task.	"Create POST /register endpoint"
DFD	Each process → 1 task. Each data store → verify DB task exists.	"Implement order processing logic"
Use Case	Each use case → verify corresponding user story task exists.	Validation / gap detection, not new tasks
Requirements	Each requirement with no matching user story → flag as "needs story".	Gap detection alert, not a coding task

14.3 Grouping Modes

Mode	How Tasks Are Grouped	Best For
By User Story	Tasks grouped under the user story they relate to	Feature-focused development, matches Agile epics
By Layer	Tasks grouped as: Database, Backend API, Frontend UI, Testing	Technical planning, estimating per-skill effort
By Feature Flow	Tasks grouped by end-to-end feature (Register, Checkout, etc)	Sprint planning, feature-based delivery

14.4 Regeneration Logic

When user clicks "Regenerate Tasks":

- Confirm dialog: "This will re-generate auto-generated tasks. Your manually added tasks will be preserved."
- Delete all tasks where is_manual = false AND project_id = current
- Re-run all extraction rules on all nodes
- Insert new generated tasks
- Manual tasks (is_manual = true) untouched

14.5 Future: AI Enhancement (Phase 3)

Rule-based extraction handles 80% of cases. For the remaining 20% (context-aware tasks, dependency ordering, effort estimation), an LLM can analyze all docs holistically and generate smarter tasks. This is a future enhancement that layers on top of the rule-based engine.

15. Tech Stack

Layer	Technology	Purpose
Framework	Next.js 14+ (App Router)	Routing, React framework
Node Editor	React Flow (@xyflow/react)	Node-based visual pipeline
Diagram Rendering	Mermaid.js	Render all diagram types from syntax
State Management	Zustand	Lightweight global state
Local Storage	Dexie.js (IndexedDB)	Persistent local data
Styling	Tailwind CSS	Utility-first CSS
Code Editor	CodeMirror 6	Syntax-highlighted Mermaid editor
PDF Export	jsPDF + html-to-image	Generate PDF documents
CSV/JSON Export	Native JS	Format task data for export
Image Export	html-to-image	Export diagrams as PNG
Deployment	Vercel	Next.js native deployment

Dexie.js Schema

```
// db.ts
import Dexie from 'dexie';

const db = new Dexie('archway');

db.version(1).stores({
```

```
    projects: 'id, name, created_at, updated_at',
    nodes: 'id, project_id, type, status, sort_order',
    nodeContents: 'id, node_id',
    edges: 'id, project_id, source_node_id, target_node_id',
    tasks: 'id, project_id, source_node_id, group_key, is_manual, sort_order',
  });
```

16. Sprint Plan & Roadmap

Sprint 1: Core Foundation (Week 1-2)

Task	Estimate
Setup: Next.js + Tailwind + Dexie + React Flow + Zustand	2-3h
Dashboard: project list, create, rename, delete	4-6h
Dexie schema + all CRUD operations	3-4h
Workspace layout: canvas left, editor panel right	3-4h
React Flow pipeline: default nodes, edges, zoom, status badges	6-8h
Basic node editor panel: open/close, load/save	4-6h

Sprint 2: Node Editors & Diagrams (Week 3-4)

Task	Estimate
Text node editors: Brief, Requirements, User Stories guided forms	6-8h
Mermaid integration: CodeMirror + live preview	4-6h
Diagram editors: Flowchart, ERD, Use Case, DFD, Sequence + starter templates	6-8h
Auto-save (debounced) + node status logic	3-4h
Soft guidance: recommended next node highlight	2-3h

Sprint 3: Task Generation Engine (Week 5-6)

Task	Estimate
Task Engine architecture: extraction rules framework	4-6h
Extraction rules: User Stories → tasks	3-4h
Extraction rules: ERD → DB tasks	3-4h
Extraction rules: Flowchart → implementation tasks	3-4h
Extraction rules: Sequence → API tasks, DFD + Use Case rules	4-6h

Task Board node UI: list, grouping toggle, CRUD	6-8h
Progressive suggestion: trigger on node save	3-4h
Regeneration logic with manual task preservation	3-4h

Sprint 4: Export & Polish (Week 7-8)

Task	Estimate
Summary node: auto-compile all nodes	4-6h
Export full project: PDF + Markdown	6-8h
Export tasks: MD checklist, CSV, JSON, Linear CSV	4-6h
Export individual diagrams: PNG	2-3h
Custom nodes: add/remove/reorder	4-6h
Polish, bug fixes, responsive design	6-8h

Future Sprints (Post-MVP)

Sprint	Focus
Sprint 5	Template Gallery (e-commerce, SaaS, API, mobile app presets)
Sprint 6	Cloud Sync (Supabase backend + auth + sync from IndexedDB)
Sprint 7	AI Task Enhancement (LLM-powered smart task generation)
Sprint 8	Collaboration (share projects, real-time editing)
Sprint 9	Drag-and-drop Diagram Editor (visual alternative to Mermaid)
Sprint 10	Direct integrations (Linear API, Notion API, GitHub Issues API)

17. Definition of Done

#	Criteria	Detail
1	Functional	Feature works per acceptance criteria in User Story
2	No Console Errors	Zero errors or warnings in browser console
3	Data Persistence	Data survives browser refresh and restart
4	Error Handling	Edge cases handled (empty states, invalid input, Mermaid syntax errors)
5	Responsive	Works on desktop (1024px+). Tablet nice-to-have.
6	Code Quality	Clean, modular, no dead code, meaningful names

7	Tested	Manual testing: happy path + edge cases passed

18. Wireframe Notes

Dashboard

```
┌──────────────────────────────────────────────────┐
│  ARCHWAY                          [+ New Project]    │
├──────────────────────────────────────────────────┤
│                                                  │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────┐   │
│   │ Toko Online  │   │ Portfolio    │   │ + New    │   │
│   │ ▓▓▓▓▓▓▓▓ 60%  │   │ ▓▓▓▓▓▓▓▓ 20%  │   │          │   │
│   │ Updated 2h   │   │ Updated 1d   │   │          │   │
│   └──────────────┘   └──────────────┘   └──────────┘   │
└──────────────────────────────────────────────────┘
```

Project Workspace (with Task Board)

```
┌──────────────────────────────────────────────────┐
│ ← Dashboard   Project Name    [Export ▼]          │
├──────────────────────────────┬──────────────────┤
│                              │  TASK BOARD       │
│  [Brief]─[Req]─[Stories]     │                   │
│     \                        │  Group: [Story▼]  │
│    [UC]─[Flow]               │                   │
│         \                    │  ■ Register Epic  │
│    [DFD]─[ERD]               │    ☐ Create form  │
│          \                   │    ☐ Validate     │
│     [Seq]─[Tasks]─[Sum]      │    ☐ Users table  │
│                              │    ☐ POST /register│
│   React Flow Canvas          │                   │
│                              │  [+Add] [Regen]   │
│                              │  [Export Tasks ▼] │
└──────────────────────────────┴──────────────────┘
```

Appendix A: Pain Point → Solution Detailed Mapping

Detail lengkap setiap pain point, dampaknya, dan bagaimana Archway menyelesaikannya.

P1: Scope Creep

Problem: Fitur nambah terus karena nggak ada batasan jelas di awal. Developer bilang "ya" ke setiap request tanpa evaluasi dampaknya terhadap timeline dan arsitektur.

Impact: Timeline molor, arsitektur jadi messy, technical debt menumpuk, developer burnout.

Archway Solution: Project Brief node mendefinisikan scope di awal. Requirements node memaksa prioritisasi (MoSCoW). Kalau mau tambah fitur, harus kembali ke Requirements node dan conscious decision: apakah ini Must, Should, Could, atau Won't? Ini menciptakan friction yang sehat terhadap scope creep.

Related Features: FR-23, FR-24

P2: Kehilangan Arah

Problem: Di tengah project, developer lupa mau ngerjain apa, apa yang sudah selesai, apa yang belum.

Impact: Kerja nggak fokus, bolak-balik antar fitur tanpa menyelesaikan satupun, burnout.

Archway Solution: Visual pipeline dengan node status (Empty/In Progress/Done) = peta progress yang selalu terlihat. Task Board node = checklist yang bisa dicek kapan saja. Kalau bingung, lihat pipeline dan Task Board.

Related Features: FR-15, FR-32

P3: Database Refactoring

Problem: Bikin tabel database asal-asalan tanpa ERD, lalu harus refactor berkali-kali saat ketemu kebutuhan baru.

Impact: Data migration yang risky, bug karena schema berubah, waktu terbuang.

Archway Solution: ERD node memaksa developer memikirkan entities, attributes, dan relationships SEBELUM menulis migration pertama. Flowchart dan Sequence Diagram juga membantu validasi apakah ERD sudah cover semua alur.

Related Features: FR-26

P4: Fragmented Tools

Problem: ERD di dbdiagram.io, flowchart di Draw.io, notes di Notion, requirements di Google Docs. Semua terpisah.

Impact: Konteks terpecah, sulit melihat gambaran besar, informasi outdated di beberapa tempat.

Archway Solution: Satu workspace: semua docs dan diagrams di satu tempat, saling terhubung lewat pipeline. Update ERD? Task Board otomatis update juga.

Related Features: FR-10, FR-11

P5: Nggak Tahu Mulai dari Mana

Problem: Developer tahu harus bikin dokumentasi tapi nggak tahu urutan yang benar. Harus bikin apa dulu? ERD atau flowchart?

Impact: Paralysis, akhirnya nggak bikin dokumentasi sama sekali dan langsung coding.

Archway Solution: Guided pipeline dengan recommended order (Brief → Requirements → Stories → Use Case → Flowchart → DFD → ERD → Sequence). Soft guidance, nggak di-lock, tapi developer tahu jalur yang ideal.

Related Features: FR-11, FR-16

P6: Dokumentasi Itu Boring

Problem: Bikin dokumentasi terasa seperti tugas kampus, bukan "building something". Nggak ada dopamine hit.

Impact: Developer malas dan skip dokumentasi, langsung vibe coding.

Archway Solution: Node-based visual experience. Setiap node yang selesai = visual progress (badge berubah hijau). Pipeline yang terlihat makin "completed" memberikan sense of progress. Bukan blank document, tapi interactive workspace.

Related Features: FR-10, FR-15, FR-22

P7: Dokumentasi Nggak Actionable

Problem: Bahkan kalau bikin dokumentasi, hasilnya cuma jadi pajangan. Nggak ada jembatan langsung ke coding tasks.

Impact: Dokumentasi terasa sia-sia, reinforcing belief bahwa "docs itu buang waktu".

Archway Solution: Auto task generation: setiap node yang diisi langsung menghasilkan tasks di Task Board. Dokumentasi bukan lagi pajangan, tapi langsung menjadi action items.

Related Features: FR-30, FR-31, FR-32

P8: Double Work (Docs → Tasks)

Problem: Setelah bikin PRD, developer harus manual translate ke task list. Ini double work yang membosankan.

Impact: Waktu terbuang, task list sering nggak lengkap karena malas translate semua.

Archway Solution: Progressive task suggestion: saat mengisi User Stories, tasks langsung muncul. Saat mengisi ERD, database tasks langsung muncul. Zero double work. Export ke MD/CSV/JSON/Linear CSV langsung siap.

Related Features: FR-30, FR-42, FR-43, FR-44, FR-45

P9: Susah Estimasi Effort

Problem: Tanpa task breakdown yang granular, sulit mengestimasi berapa lama project akan selesai.

Impact: Deadline missed, client kecewa, developer stress.

Archway Solution: Task generation dari semua diagram types menghasilkan breakdown yang granular. Dari ERD muncul DB tasks, dari flowchart muncul implementation tasks, dari sequence muncul API tasks. Total tasks = rough effort estimate.

Related Features: FR-31, FR-33

P10: No Source of Truth

Problem: Saat coding, nggak ada satu tempat yang bisa dirujuk untuk "apa requirement-nya" atau "gimana flow-nya".

Impact: Developer bikin asumsi yang salah, fitur nggak sesuai requirement, rework.

Archway Solution: Archway = single source of truth. Summary node compile semuanya. Exportable sebagai PDF (bisa diprint/buka di tab) atau Markdown (bisa commit ke repo). Selalu ada anchor.

Related Features: FR-27, FR-40, FR-41

▶ **This document is your anchor.** Every decision has been thought through. Every pain point has a solution. Now it’s time to build. Start Sprint 1.